Unified Picker: Multiplicative Score Formula

Regime A (default), with Regime B extension

sharpsat-separator (Phase 1)

1 Score formula (Regime A, $\gamma = 0$)

For each active branch candidate x in the current sub-component, the picker computes

$$S(x) \;=\; \text{base}(x) \cdot \text{boost}(x), \qquad \text{pick} \;=\; \arg\max_{x \in \text{candidates}} S(x).$$

Type-pure base score

$$\text{base}(x) \;=\; \begin{cases} w_v \cdot \max\big(\varepsilon,\; \text{freq}(v) + 10\;\text{activity}(v) + w_c \cdot \text{cheap}(v)\big) & x = v \text{ (variable)} \\ w_C \cdot \sigma\big(\beta\,(L(C) - m)\big) & x = C \text{ (clause)} \end{cases}$$

The clause-base parameters β and m control the shape of the length-sigmoid.
The length-decayed clause-density proxy:

$$\text{cheap}(v) \;=\; \sum_{C \ni v,\ \text{active}} 2^{-\alpha_d\,|C|_{\text{active}}} \;+\; \text{binPart}(v) \cdot 2^{-2\alpha_d},$$

where $\text{binPart}(v)$ is the count of length-2 clauses still actively constraining v under the current partial assignment.

Multiplicative separator boost

$$\text{boost}(x) \;=\; 1 \;+\; \alpha \cdot \exp(-\lambda \cdot \text{rel}_k) \cdot \mathbf{1}[x \in \text{sep}],$$

$$\text{rel}_k \;=\; \frac{\#\{\text{active separator elements}\}}{\#\{\text{active variables in comp}\}}.$$

Above we use the bold-one indicator

$$\mathbf{1}[P] \;=\; \begin{cases} 1 & \text{if predicate } P \text{ is true} \\ 0 & \text{otherwise.} \end{cases}$$

Properties

- $\text{base}(x) > 0$ for every active candidate (the ε floor on var-base prevents the degenerate all-zero case), hence $S(x) > 0$ always. No "no candidate found" edge case.
- $\text{boost}(x) \in [1,\; 1 + \alpha]$ — never penalizes a candidate; only multiplicatively rewards separator membership.

- Within a type, ranking is monotone in base: at a given node, boost is the same across all candidates of one type that share the same separator-membership status.
- Cross-type competition (var vs. clause) happens entirely via the ratio $w_v\,\overline{\text{base}_v} \;:\; w_C\,\overline{\text{base}_C}$. The defaults $w_v = 1, w_C = 7$ are calibrated so a length-4 clause ($\sigma(1) \approx 0.731$) has base ≈ 5, comparable to a typical active variable's freq +20 activity.

2 Regime B extension (not yet enabled; $\gamma > 0$)

When the cascade boost is enabled, the multiplier acquires a third additive component proportional to how strongly the variable's BCP-cascade dominates its peers in the current sub-component:

$$\text{boost}_{\text{Regime B}}(v) \;=\; 1 \;+\; \alpha \cdot e^{-\lambda\ \text{rel}_k} \cdot \mathbf{1}[v \in \text{sep}] \;+\; \gamma \cdot \max\Big(0,\ \frac{\text{depth}(v)}{\text{depth}_{\text{med}}} \;-\; 1\Big),$$

$$\text{depth}(v) \;=\; \log_2 \max\big(1,\ \texttt{computeCascadeScore}(v)\big),$$

with $\text{depth}_{\text{med}}$ estimated as the median over a uniform random sample of $K = 30$ active variables in the comp (full-set median is prohibitively expensive per pickBranchTarget call).

The boost is *relative to the local median*: vars at or below the median contribute 0, and the contribution grows linearly above it. A var with $\text{depth}(v) = 2 \cdot \text{depth}_{\text{med}}$ adds γ; a var with $5 \cdot \text{depth}_{\text{med}}$ adds 4γ.

This calibrates automatically per-node: cascade-rich components reward only top-cascade outliers; cascade-poor components recognize modest cascades as locally exceptional. No absolute threshold.

For clauses, $\text{boost}_{\text{cas}}(C) = 0$ — the cascade boost applies only to variables.

3 Symbol table

Symbol	Code knob	Default	Role
w_v	`picker_var_weight`	1.0	Variable-base anchor.
w_C	`picker_clause_weight`	7.0	Clause-base anchor (calibrated against typical var-base on real instances; see §5).
w_c	`cheap_score_weight`	0.0	Cheap-score addend weight inside variable base; default off.
ε	`picker_base_floor`	0.01	Floor on variable base, keeps $S(x) > 0$.
β	`clause_length_steepness`	1.0	Sigmoid slope in clause base.
m	`clause_length_midpoint`	3.0	Sigmoid midpoint in clause base.
α	`picker_alpha`	15.0	Separator-boost gain; sep candidates at $\text{rel}_k = 0$ get a $1 + \alpha = 16\times$ multiplier.
λ	`picker_lambda`	5.0	Separator-boost decay rate; smooth fade as separator grows relative to comp.
γ	`picker_gamma`	0.0	Cascade-boost gain; 0 in Regime A.
α_d	`stage0_length_decay`	auto	Cheap-score length decay; auto-tuned at preprocess.

4 Definitions

The variable-base components are defined as follows.

Frequency. freq(v) is the number of active original clauses containing the variable v (either polarity), counted at the current node:

$$\text{freq}(v) \;=\; \#\big\{\, C \in \text{active original clauses} \;:\; v \in \text{vars}(C) \,\big\}.$$

"Active" here means the clause is not yet satisfied or removed under the current partial assignment, and contains at least one unassigned literal. This is the length-agnostic clause-count proxy for v's structural prominence.

Activity (VSIDS). activity(v) is the sum over both polarities of the recency-weighted "conflict involvement" counter from the *Variable State Independent Decaying Sum* heuristic:

$$\text{activity}(v) \;=\; a^{+}(v) \;+\; a^{-}(v),$$

where $a^{+}(v)$ and $a^{-}(v)$ are the per-literal activity scores (`literal(LiteralID(v,sign)).activity_score_` in the code) maintained as follows during search:

- All activities start at 0.
- On every conflict, each literal that participated in the conflict analysis (UIP-derived learned clause, etc.) has its activity *bumped* by an increment Δ_{bump}.
- Every N decisions (in this codebase, every 128), all activities are scaled by a decay factor $0 < d < 1$ (`decayActivities()` in `solver.cpp`).

This produces a recency-biased measure of how often the literal has recently appeared in conflicts. The factor 10 in the variable-base formula scales the summed two-polarity activity into the same magnitude range as freq ($\sim$ 0–100 in typical search, versus freq $\in$ 0–10); empirical default in `Solver::scoreOf`.

Clause length weighting for clause branching

Binary clauses part

$$\text{binPart}(v) \;=\; \#\big\{C = (\ell_v, \ell_b) \in \mathcal{B} \;:\; \ell_v \text{ is a literal of } v,\ \ell_v \text{ unassigned},\ \ell_b \text{ unassigned}\big\}$$

is the count of *live* (currently neither satisfied nor unit) binary clauses in $\mathcal{B}$ containing v. Here $\mathcal{B}$ is the set of binary clauses stored in `binary_links_` — original binaries plus in-scope learned binaries. Equivalently: the number of length-2 clauses still actively constraining v under the current partial assignment.

The standard sigmoid

$$\sigma(z) \;=\; \frac{1}{1 + e^{-z}},$$

- m (*midpoint*, default 3) is the active clause length at which the sigmoid output equals $\frac{1}{2}$. Below m, the base shrinks toward 0; above m, it saturates toward 1.

- β (*steepness*, default 1) controls how sharply the transition occurs around m. Small β ($\to 0$) flattens the curve and treats all clause lengths almost equally; large β ($\to \infty$) approaches a hard step at $L = m$. Concrete values at $m = 3$:

L	$\beta = 0.3$	$\beta = 1$ (default)	$\beta = 5$
2	0.43	0.27	0.007
3	0.50	0.50	0.50
4	0.57	0.73	0.99
5	0.65	0.88	≈ 1

  Default $\beta = 1$ gives a moderate gradient: length-4 scores $\sim 1.5\times$ length-3, length-5 scores $\sim 1.8\times$ length-3. This implements a *soft* preference for longer clauses (longer $\Rightarrow$ more lits pinned in the negate-all branch $\Rightarrow$ deeper BCP cascade), without forcing a hard threshold.

5 Calibration rationale

The default $w_C = 7$ is not arbitrary. On instances with freq(v) $\approx$ 3–5 (typical for sparse 3-SAT with $\sim$1:1 var:clause ratio) and activity(v) ≈ 0 (early in search, before learning has fired), the variable base is in $[3, 5]$. A length-4 clause has $\sigma(\beta(4-3)) = \sigma(1) \approx 0.731$. Setting $w_C \cdot 0.731 \approx 5$ gives $w_C \approx 7$, equating clause base to typical var base for $L = 4$ clauses.

The default $\alpha = 15$ was chosen so that a separator candidate at $\text{rel}_k = 0$ gets a $16\times$ multiplier on its base, dominating any non-separator candidate by a wide margin. This approximates the legacy `-sepVarBias` mode's near-strict "consume separator first" preference.

These defaults are starting points for the manual sensitivity sweep prescribed in docs/unified_picker_redesign.md (Phase 4). The doc's originally-suggested ranges (e.g. $w_C \in [0.3, 3.0]$) were two orders of magnitude too small relative to the sigmoid-clamped clause-base range $(0, 1)$ once real var-base magnitudes were measured; the calibration here resolves that.

6 Worked example (Regime A)

A sub-component with $N_{\text{active vars}} = 80$, separator size 3 (2 vars +1 clause):

$$\text{rel}_k = \tfrac{3}{80} = 0.0375, \qquad \alpha \cdot e^{-\lambda \cdot 0.0375} = 15 \cdot e^{-0.1875} \approx 15 \cdot 0.83 \approx 12.4.$$

A separator candidate gets a multiplier $1 + 12.4 = 13.4\times$ on its base. With default weights, a length-3 separator clause has base $w_C \cdot \sigma(0) = 7 \cdot 0.5 = 3.5$, yielding $S = 3.5 \cdot 13.4 \approx 47$. A typical non-separator var with base ≈ 5 yields $S = 5$. The separator clause wins decisively.

When the separator inflates to 20 elements ($\text{rel}_k = 0.25$):

$$\alpha \cdot e^{-1.25} \approx 15 \cdot 0.287 \approx 4.3,$$

the multiplier drops to $5.3\times$, and the separator clause's $S \approx 18.6$ now competes much more closely with high-base non-separator candidates — the smooth handover regime the formula was designed to produce, with no thresholds.